



重慶理工大學

实验报告

课程 嵌入式系统及应用

题目 USART 串行通信

院系名称 计算机科学与工程学院

专业 物联网工程 班级

学生姓名 学号

任课教师

时 间

一、实验设备与软件环境

设备:

- ① PC 电脑, 至少需要 2 个 Type-A 的 USB 插口;
- ② ALIENTEK MiniSTM32 实验套件, 目标芯片为 STM32F103RCT6;
- ③ STLink V2 下载器、USB 转 TTL 串口模块和杜邦线。

环境: macOS 26.3; probe-rs/cargo-embed; VSCode Serial Monitor。

二、实验报告任务

(1) 串口查询方式按键消息发送: 该方式要解决 3 个按键状态变化时发送一次消息 (keydown、keyup), 而无变化时不发送;

(2) 串口运算程序设计

十进制数算数逻辑运算+、-、*、\、%、&、|运算命令, 要求如下所示。

运算练习:

如 PC 发送"123+4000=?\r\n", 则 STM32 向 PC 串口返回"123+4000=4123\r\n"。

三、主要程序分析

3.1 USART1 初始化与收发

setup_clock_gpio_usart() 先开启 AFIO、GPIOA、GPIOC 和 USART1 时钟, 然后关闭 JTAG、保留 SWD, 使 PA15 可以作为 KEY1 输入。USART1 使用 PA9 作为 TX、PA10 作为 RX, BRR 写入 0x45, 对应 8MHz 外设时钟下约 115200 波特率。

Uart1::write_byte() 通过读取状态寄存器 SR 的 TXE 位判断发送数据寄存器是否为空, 空了之后再写一个字节写入 DR。read_byte() 检查 RXNE 位, 若接收数据寄存器非空, 就读取 DR 的低 8 位并返回。Write trait 只负责把字符串逐字节发出。

3.3 按键状态变化处理

KEY0 接 PC5, KEY1 接 PA15, 二者按下为低电平; WK_UP 接 PA0, 按下为高电平。Button 结构保存 stable、last_raw、count 三个值, 连续读到同一原始电平达到 DEBOUNCE 后, 才认为按键状态稳定改变。状态改变后调用 send_key_message() 发送 KEY0 keydown、KEY0 keyup 等消息。

3.4 串口输入与运算解析

handle_serial_input() 持续读取已经到达的字节, 普通字符保存到 input_buffer, 遇到 \r 或 \n 时调用 process_line()。如果串口调试工具没有发送回车换行, 主循环会用 input_idle_count 判断空闲时间, 超过 WAIT_SERIAL 后也处理当前缓冲区, 避免输入后一直没有响应。

calculate() 采用简单顺序解析: 先读左操作数, 再读一个操作符, 最后读右操作数。若表达式合法, 则根据操作符返回整数结果; 除数为 0、缺少操作符、缓冲区溢出或包含非法字符时返回 ERR。当前工程支持 +、-、*、/、\、%、&、|, 其中 / 和 \ 都按整数除法处理。

四、实验连线

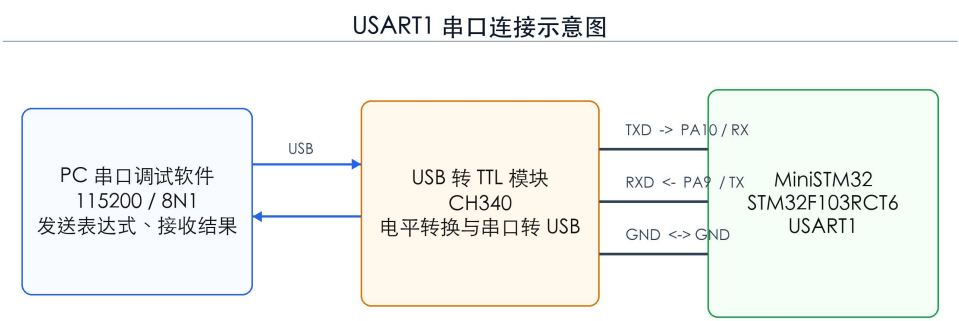


图 4-1 USART1 串口连接示意图

本实验使用 USART1 与 PC 串口调试软件通信。STM32F103RCT6 的 PA9 为 USART1_TX，通过跳线帽连接到 USB 转 TTL 模块的 RXD；PA10 为 USART1_RX，连接到 USB 转 TTL 模块的 TXD。

连接对象	MiniSTM32 端	USB 转 TTL/PC 端	说明
串口发送	PA9 / USART1_TX	RXD	STM32 向 PC 发送 keydown、keyup 和计算结果。
串口接收	PA10 / USART1_RX	TXD	PC 向 STM32 发送表达式。
下载调试	SWD	STLink	下载和复位程序。
按键输入	PC5、PA15、PA0	板载 KEY0、KEY1、WK_UP	不需要外接按键线。

五、程序流程图

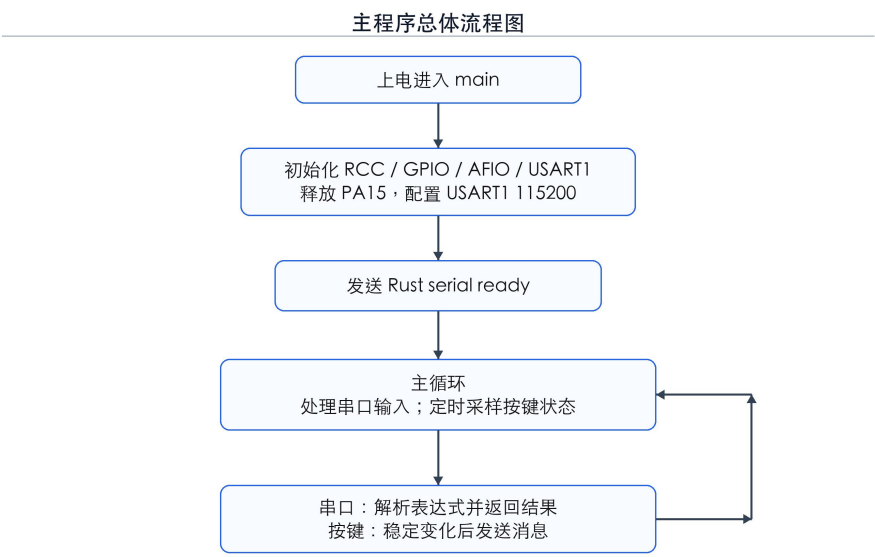


图 6-1 主程序总体流程图

主程序先完成外设初始化并打印 ready 信息，之后一直在无限循环中处理两个任务：串口接收和表达式计算、按键状态采样和消息发送。

串口输入与表达式计算流程图

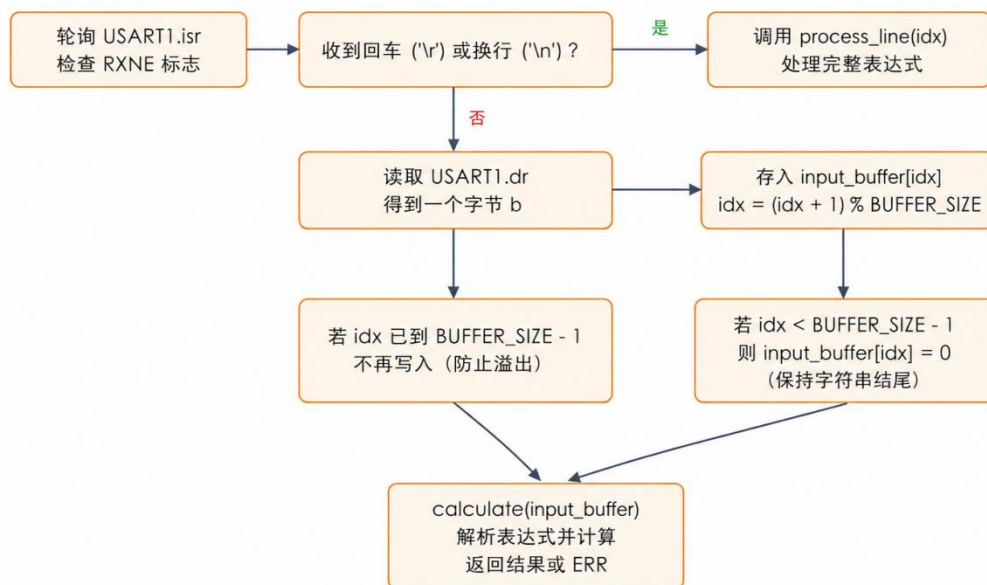


图 6-2 串口输入与表达式计算流程图

串口处理函数尽量一次读完已经到达的字节。收到回车换行时立即计算；没有回车换行但输入已经空闲一段时间时，也会处理当前表达式。

按键状态变化发送流程图

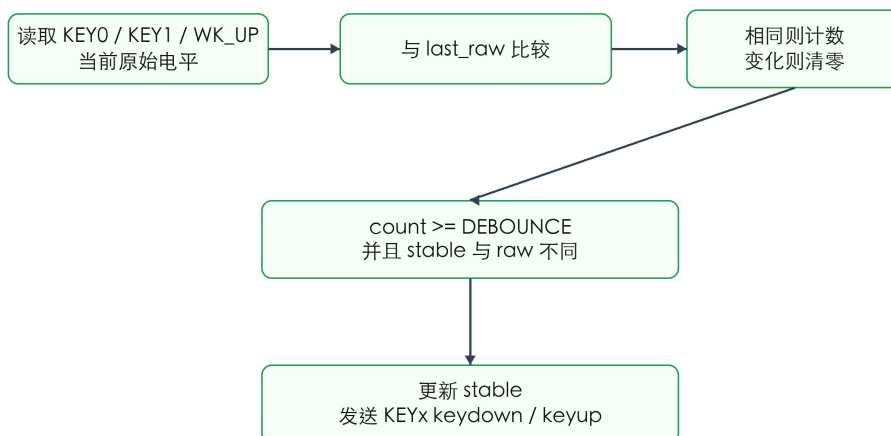


图 6-3 按键状态变化发送流程图

按键处理不直接把一次读取结果当作最终状态，而是先记录连续稳定次数。只有稳定状态与上次状态不同，才发送 keydown 或 keyup。

六、程序调试及结果分析

6.1 编译与下载结果

烧录后使用 VSCode Serail Monitor 监听端口, 设置波特率为 115200. 开发板复位后 PC 端可看到 Rust serial ready, 说明 USART1 已正常初始化并能从 PA9 发处数据。

6.2 实验现象

测试项目	操作	观察现象	结果分析
KEY0	按下/松开 KEY0	发送 KEY0 keydown 和 KEY0 keyup	PC5 上拉输入, 按下为低电平, Button 稳定后只发送一次。
KEY1	按下/松开 KEY1	发送 KEY1 keydown 和 KEY1 keyup	程序关闭 JTAG 后释放 PA15, 因此 KEY1 能作为普通输入。
WK_UP	按下/松开 WK_UP	发送 WK_UP keydown 和 WK_UP keyup	PA0 下拉输入, 按下读到高电平。
加法	输入 123+400	返回 123+400=523	左操作数、操作符和右操作数解析正确。
乘法	输入 12*3	返回 12*3=36	普通算术运算正常。
除零	输入 1/0	返回 ERR	calculate() 中对除数为 0 做了保护。

问题 输出 终端 串行监视器 端口 调试控制台

十 打开其他监视器

监视模式 Serial 查看模式 文本 端口 /dev/tty.usbserial-110 - 波特率 115200

数据位 8 停止位 1 奇偶校验 无 ☒ DTR ☒ RTS 选择日志文件目录

Rust serial ready
---- 已发送 utf8 编码消息: "123+4000" ----
123+4000=4123
---- 已发送 utf8 编码消息: "123+400" ----
123+400=523
---- 已发送 utf8 编码消息: "12*3" ----
12*3=36
---- 已发送 utf8 编码消息: "12*3\r\n" ----
12*3=36
WK_UP keydown
WK_UP keyup
KEY1 keydown
KEY1 keyup
KEY0 keydown
KEY0 keyup
WK_UP keydown
KEY1 keydown
WK_UP keyup
KEY1 keyup
---- 已发送 utf8 编码消息: "1/0\r\n" ----
ERR
|

6.3 调试过程中重点问题及处理方法

(1) 部分串口调试工具发送内容时不会自动附加 `\r\n`。若程序只等待回车换行, 输入表达式后可能没有返回。本工程增加 `input_idle_count`, 当收到一段字符后长时间没有新字节, 也把当前缓冲区作为一行处理。

(2) debug 版本在 115200 波特率下运行较慢, 连续接收表达式时可能丢字节, 表现为返回 ERR。处理方法是使用 release 版本下载验证, 最终以 `cargo build --release` 和 `cargo embed/probe-rs` 下载为准。

(3) 按键机械抖动可能造成一次按下多次发送。Button 中用 `last_raw`、`count` 和 `stable` 做简单去抖, 连续多次读到同一状态后才报告一次状态变化。

七、思考题

(1) printf 函数在我们调试程序时有哪些帮助? printf 与 SWD 调试相比各自能调试(显示) 哪些信息、如何显示中间变量等信息、有哪些特点?

printf 类串口输出适合快速观察程序运行到哪里、当前变量值是多少、某个分支是否进入。它的优点是直观, 不需要一直连接调试器, 程序运行状态能直接留下文本记录。缺点是会占用一个串口。

SWD 调试可以暂停 CPU、单步执行、查看寄存器、内存、外设寄存器和调用位置, 也可以设置断点观察某个变量在某一行代码前后的变化。SWD 的缺点是调试时会改变程序的连续运行状态, 对串口接收这类实时过程有时会带来额外影响。

(2) 如何理解将收到的单字符原样转发语句“USART1->DR = USART1->DR;”中等号两边是同一个寄存器?

右边读取 `USART1->DR` 时, 是从接收数据寄存器取出刚收到的字节; 左边写入 `USART1->DR` 时, 是把该字节放入发送数据寄存器, 等待 USART 硬件移位发送出去。STM32 的同一个 DR 地址在读和写时连接到不同的数据通路, 所以可以用这种写法实现“收到什么就发回什么”。

(3) 在完整数据包使用回车换行结束是一种简单通讯格式或叫协议, 那么在串口通讯中还有哪些常见的标准软件通讯协议?

常见串口软件通讯协议有 Modbus RTU、Modbus ASCII、AT 命令格式、NMEA 0183、SLIP、PPP、XMODEM、YMODEM、ZMODEM 等。它们都会在普通字节流上规定帧格式、结束方式、校验方式或应答方式。

例如 Modbus RTU 常用于工业设备, 使用地址、功能码、数据和 CRC 校验组成一帧; AT 命令常用于蓝牙、WiFi、蜂窝模组, 通常以字符串命令和回车换行结束; XMODEM/YMODEM/ZMODEM 常用于串口文件传输, 包含分包、序号、校验和重传。相比本实验的“表达式 + 回车换行”, 这些协议对错误检查、设备寻址和传输可靠性要求更高。